NimbusPay Ticket Triage — Fine-Tuning Report

Base model `Qwen/Qwen2.5-1.5B-Instruct` (1.54B, under the 4B cap), **16-bit · LoRA via PEFT** `r=16, α=32`, all attn+MLP projections → **18.46M trainable, 36.9% of the 50M cap · Stack** `transformers.Trainer` + `peft`, hand-written loss mask · Free Colab **T4**, peak **11.82 GB**, **19.9 min**.

Headline: dev exact match 90.0%, mean field accuracy 97.9%, JSON validity 100% *(n=200), from 0.0% / 0.0% / 0.0% un-tuned on the same prompt. The submitted adapter is this run; the best experimental arm reached* ***93.0%*** *(§4, E6).*

1. Audit

Method. I encoded `SCHEMA.md` as an executable validator and ran it on `dev.jsonl` *first*. Dev is stated clean, so it is a **control**: a flag there means my validator is wrong, not the data. I iterated to **zero flags on all 200 dev rows**, then pointed it at train — so every train flag is a real defect, not a misreading. It caught two bugs of mine: my repeat-contact regex fired on 16 clean rows because quoted `>` replies contain "already received a resolution"; and `phir bhi` (`dv-00118`) means "even so", not "again".

1,445 rows in → 1,350 out. 95 dropped, 191 fixed, 388 lines in `cleaning_log.csv`.

#	Defect	Rows	Action	Found by
D1	Label JSON truncated mid-string	25	**drop**	`json.loads` fails; unrecoverable
D2	Placeholder bodies (`test`, `asdf`, one empty)	10	**drop**	length floor; dev min 44 chars, these 0–13
D3	Legacy key `prio`, values `low/medium/high`	70	fix (rename+remap)	key-set histogram: two shapes
D4	`txn_date` as `DD/MM/YYYY`	30	fix	format regex; **all 30 inside D3**
D5	`txn_id` unnormalized (`np 81633 47892`)	20	fix	`NP\d{10}`; none inside D3
D6	`amount_paise` stored in **rupees**	20	fix (×100)	matches a rupee figure, not its paise value
D7	`priority` violates the spec ladder	62	fix (recompute)	rule engine, after D3/D6
D8	`needs_human` violates the spec	21	fix (recompute)	rule engine, after priority cascade
D10	Exact duplicate ticket texts	60	**drop**	text hash; **labels agree** — reweighting, not corruption

D3 mattered most. The obvious move on 70 rows keyed `prio` is to recompute priority — discarding 70 *correct* labels. The values are `low/medium/high`, and against what the v2 ladder derives for the same row the mapping is perfect with **zero exceptions** (`low`→`P3` ×44, `medium`→`P2` ×15, `high`→`P1` ×11). `SCHEMA.md` calls itself "v2, current": this is a v1 export, so the fix is a value remap. The 30 `DD/MM/YYYY` dates falling *entirely* inside these rows confirmed one legacy exporter, not three faults.

Fix order is load-bearing. D6 must precede D7. `tr-00382` stores ₹46,094 as `46094` paise (₹460.94), pushing it under the ₹5,000 P2 threshold and making a *correct* `P2` look like a violation; recomputing priority first would have "fixed" three correct labels into wrong ones. Naively D7 is 135 rows; in the right order, 62. The cleaned set then passes the validator dev passes.

Rejected. *Urgency drove the over-escalation:* 33 rows are `P1` where the ladder gives `P3`, several shouting `URGENT!!` — but urgency markers appear at **24% vs 21%** of other rows, so I correct by rule and claim nothing about origin. *A bad annotator batch:* across ten id ranges of ~145, every defect class spreads evenly (1.4%–7.2%). **Checked, nothing found:** distractor amounts (balance/limit/fee) mistaken for the claim — **0 rows**, dev *and* train; fraud-worded tickets labelled otherwise — **0**; `language` corruption — none.

2. Baselines and pipeline

Identical inference contract throughout: verbatim `system+user`, Qwen chat template, greedy, `max_new_tokens=200`, `.strip()` as the only post-processing.

Run (dev, n=200)	json_valid	exact_match	mean_field_acc
B1 — base, fixed prompt	0.0%	0.0%	0.0%
B2 — base, **full rules in prompt** *(reference only)*	97.5%	0.0%	42.9%
Fine-tuned	**100.0%**	**90.0%**	**97.9%**

B2 is the informative one. The full rulebook in the prompt fixes *format* (json_valid 0 → 97.5%) and buys almost no *task* accuracy: exact match stays **0.0%**, and `priority` scores **0.0%** across all 200 rows while `amount_paise` reaches 81.5%. The gap was never "the model cannot see the rules" — it extracts fields once told the schema and still cannot apply the ladder. That is the evidence that fine-tuning, not prompting, is the lever.

B1's 0% is real, and I nearly mis-reported it. My first run gave `json_valid=0%` with every generation `!!!!!!!!` — not a model result: `!` is token id 0, and repeated token 0 is the signature of NaN logits, here from `attn_implementation="eager"` overflowing fp16 on Qwen2's activation outliers. With `sdpa` the 0% is *genuine*: coherent JSON in ```` ```json ```` fences with an invented schema (`status`, `ticket_id`). `load_base` now asserts finite logits.

Chat template. Qwen2.5 is ChatML with a real `system` role, so the fixed prompt passes through untouched; training uses the *same* `apply_chat_template` call as inference — no train/serve skew.

Loss on the right tokens. I left TRL's `SFTTrainer` deliberately: it masks by string-matching a `response_template`, and a silent miss trains the model to reproduce tickets with a healthy-looking loss curve and no error. I build the mask directly — everything up to and including `<|im_start|>assistant\n` is `-100` — and **assert** the unmasked span decodes back to exactly the label. Verified live: span `133..214`, **82 of 215 tokens**.

Sequence length, from evidence. Median **168** tokens, p90 **214**, p99 **1936**, max **2215** — bimodal, with **38 rows (2.8%)** carrying quoted `>` history. The label sits at the *end*, so right-truncation amputates the **target**; `MAX_SEQ_LEN=2240` covers the longest. *Rejected:* dropping long rows — 11 of 400 test rows are long. It has a cost: at `bs=4` the T4 OOM'd at 13.84/14.56 GiB because `group_by_length` packs the outliers into one batch. Fixed with `bs=2 × accum=8` (**effective batch unchanged at 16**) + gradient checkpointing → peak **11.82 GB**.

Capacity. `r=16` → 18.46M. The cap binds: `r=32` is 36.9M and `r=64` **is 73.9M, breaching 50M**, so the rank sweep is 16 vs 32. LR 2e-4 cosine, 3% warmup, 3 epochs = **255 steps**; mean training loss 0.064, the logged curve falling 0.491 (step 25) → 0.003 (step 250). fp16 only — T4 is Turing, no bf16, asserted at load.

3. Results and error analysis

Per-field (fine-tuned, dev): `amount_paise` 100% · `txn_id` 100% · `channel` 100% · `language` 100% · `txn_date` 99.0% · `category` 98.0% · `needs_human` 94.5% · `priority` **91.5%**.

Per-category: `other` / `kyc` / `account_access` **100%** · `offers` 94.4% · `refund` 89.1% · `payment_failed` 81.1% · `fraud` **80.0%**.

The three categories at 100% are exactly the three where the spec forces every transaction field to `null`. Every remaining error sits where the ladder actually fires: the model has finished learning *extraction* and now fails only at *applying rules to what it extracted*.

Root cause vs cascade. The spec is a dependency graph (`needs_human` ← `priority` ← `category+amount_paise`), so one upstream error surfaces as three wrong fields. Classifying by root cause, and calling any field correct *given the model's own upstream prediction* a cascade:

```
20/200 dev rows have at least one error
ROOT CAUSES                                          CASCADES
  9  priority: near a rupee threshold                  11  needs_human: follows its own wrong priority
  5  priority: ladder misapplied                        3  priority: correct for its own wrong inputs
  4  category: fraud confusions
  2  txn_date: relative-date arithmetic off by +1 day
total wrong fields 34 = 20 root + 14 cascade
```

Naive counting reports "`needs_human` wrong 11 times"; **zero** are independent — all follow a wrong priority. 34 wrong fields are **20 real mistakes**, and **9 of 20 (45%) are the ₹5,000/₹50,000 boundary**. The classifier is validated by 12 unit tests that perturb a gold label in a known way and assert the diagnosis; all 12 pass. One ordering bug surfaced that way — the generic "invented a date" bucket sat *before* the quoted-email check, hiding the cause.

Test set (400 rows). 400/400 parse, 0 with a wrong key set; the adapter reloaded from disk regenerated 12 random rows **12/12 identical** to `predictions_test.jsonl`. The test half is visibly harder: **7 rows (1.75%) carry values outside the schema's domain** versus **zero on dev** — `priority: "PUR"`, `channel: "apple_pay"`, four bare reference numbers as `txn_id`, and `txn_date: "2026-07-00"` (the ticket says "6 din pehle" against `Received 2026-07-06`: the model subtracted 6 from the *day digits* instead of doing date arithmetic). Repair is forbidden, so these stand; they size the dev→test gap honestly.

4. Controlled experiments

One thing changes per row; everything else is E0's configuration. Dev, n=200.

Arm	What changed vs E0	exact	field acc	peak GB	train min
E0	reference — cleaned data, 16-bit, r=16, lr 2e-4, 3 epochs	90.0%	97.9%	11.82	19.9
E1a	**raw** `train.jsonl`, no audit or cleaning	79.0%	96.6%	11.82	21.0
E3a	**2 epochs** instead of 3	74.0%	95.5%	11.82	12.7
E5	**dependency-ordered** output keys *(own idea)*	89.0%	97.9%	11.82	19.0
E6	**+720 counterfactual minimal pairs** *(own idea)*	**93.0%**	**98.8%**	11.99	43.2
E2a	4-bit QLoRA	*not run — see below*			

E1a: the audit is worth 11 points. Removing the clean step costs **11.0** exact match — and not as a sampling artefact: the raw arm's training loss was higher at *every* logged step, which is what 25 truncated labels and 62 spec-violating priorities look like from inside the optimiser.

E3a refuted my own prediction. I expected 2 epochs within ~2 points, because the loss curve is visibly flat by then (final training loss **0.0097**, which reads as converged). Dev says **–16.0**. Training loss here measures *format* memorisation, which saturates early; the rule ladder is still being learned long after the curve flattens — only a held-out measurement shows it.

E5 did nothing, and that is the useful part. Emitting `amount_paise` *before* the `priority` that depends on it scored **89.0%** — a 1-point loss, i.e. noise. My reasoning: the model applies two rupee thresholds without writing the amount down. Wrong about the bottleneck: the amount is in the **input**, which self-attention reads in full from every output position regardless of key order. Chain-of-thought pays when a value must be *derived*; here it need only be *looked up*, and `amount_paise` was already 100%. The null sharpened the diagnosis for E6.

E6 is the best arm: +3.0 exact, +0.9 field. If the residual failure is the *threshold comparison* rather than the plumbing around it, the fix is more examples near the boundary: 720 oracle-generated minimal pairs took ₹5,000/₹50,000 coverage from **106 → 264** rows. Cost 2.2× the training time for 1.5× the data, +0.17 GB.

E2a (16-bit vs 4-bit QLoRA) was not run. It is coded and was launched, but the runtime disconnected mid-run and the free-tier GPU quota was exhausted before a restart. I report the gap rather than estimate the row; it is the one mandated comparison missing.

5. What I would do next

1. **Promote E6 to the final model.** It is 3 points better, and `adapter/` holds E0's. I did not swap it in because the brief requires `notebook.ipynb` to reproduce the *submitted* model top to bottom — that means re-executing the notebook against the counterfactual set, ~70 min unbroken, and the quota is gone. Cheapest 3 points available.
2. **Extend the oracle to** `txn_date`, the one trigger E6 does not cover: shift the `Received` header, restate the relative phrase, recompute by rule — `2026-07-00` is digit arithmetic standing in for date arithmetic.
3. **Finish the sweep** — the QLoRA arm above, plus `r=16` vs `32` (`r=64` is 73.9M and breaches the 50M cap) and lr 1e-4 vs 2e-4.
4. **A second seed** before believing any gap under ~2 points (E5 sits inside that band), and oversampling `fraud` — 6.2% of rows, weakest category at 80.0%, and it forces two other fields.

6. AI tools

I used **Claude (Claude Code)** throughout, as I would on the job: it wrote the validator, cleaner, notebook, loss-mask assertion and experiment harness, drove the Colab run through the browser, and drafted this report from real output. The *method* — encode the spec, use dev as a control, fix in dependency order — and both own ideas came out of the back-and-forth; D3 came from reading the mismatch table it produced. **Where I overrode it:** it reached for TRL's `SFTTrainer` and I moved to a hand-written mask, so masking is an assertion rather than a default; its "urgency drives over-escalation" hypothesis measured out at 24% vs 21% and is reported rejected; its first error-attribution ordering had the bug the unit tests caught.